University of British Columbia
Electrical and Computer Engineering
Introduction to Microcomputers EECE259

Lecture 21: Serial Peripheral Interface (SPI)

Dr. Jesús Calviño-Fraga P.Eng.
Department of Electrical and Computer Engineering, UBC
Office: KAIS 3024
E-mail: jesusc@ece.ubc.ca
Phone: (604)-827-5387

June 18, 2014

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Godfrey Hounsfield
Nobel Prize In Medicine 1979

Electrical Engineer. Got the award for the invention of X-ray Computed Tomography (CT)

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Objectives

- Recognize the advantages and disadvantages of using serial buses.
- Understand the SPI signals and data format.
- Connect SPI devices to the microcontroller or microcomputer.
- Understand the four different signals modes of SPI.
- Generate SPI signals and interface to devices using general purpose pins.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Serial Bus Vs. Parallel Bus

- Uses less wires and pins: physically smaller devices!
- Requires more elaborate hardware, but not a problem nowadays.
- Less crosstalk.
- Clock required! All transfers are synchronous.
- <u>Most inexpensive microcontrollers do not come with a true parallel bus anymore!</u>

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

The Serial Peripheral Interface (SPI) port (SEP in the textbook)

- Used to add multiple external I/O devices or functions to the microcontroller.
- SPI devices are **extremely** simple to connect to most modern microcontrollers/microcomputers.
- SPI devices are usually smaller and cheaper than their parallel bus equivalents.
- Although access is slower than with parallel devices, communication speed is fast enough for most applications. Software overhead is not too bad either!
- Many SPI devices available. Go to Digikey and search for 'SPI'. (The long list in the textbook is a bit outdated)
- SPI article in Wikipedia is excellent!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

SPI Synchronous Data Communication Data Format

- Data length can be any size but optimized for 8 bits on most 8-bit microcontrollers.
- Clock is transmitted in a separate wire. The polarity of the clock and its phase is selectable.
- Data and clock are transmitted at the same time. Data is also transmitted and received at the same time!
- Most microcontrollers, including the 8051, can be setup as master or slaves to implement master/slave networks.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Standard SPI Signals

- Master in / Slave out (MISO)
- Master out / Slave in (MOSI)
- Serial Clock (SCK)
- Slave select (SS')

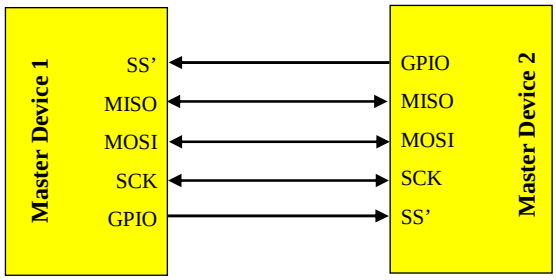
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

SPI Wiring modes

- Master: The device initiates and controls the flow of data. Can be configured as:
  - Multi-master.
  - Three wire single master.
  - Four (or more) wire master.
  - Four wire master with daisy-chained slaves.
- Slave: The device needs to be selected, and the clock is provided to it.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

SPI Wiring Modes

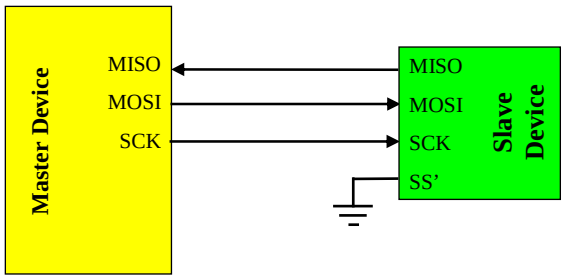


Multiple-Master Mode Connection Diagram

GPIO: General Purpose Input/Output. Any available pin in the 8051 will do!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

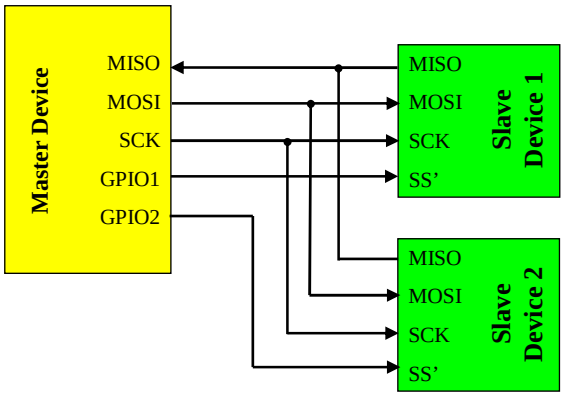
SPI Wiring Modes



3-Wire Single Master and 3-Wire Single Slave Mode Connection Diagram

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

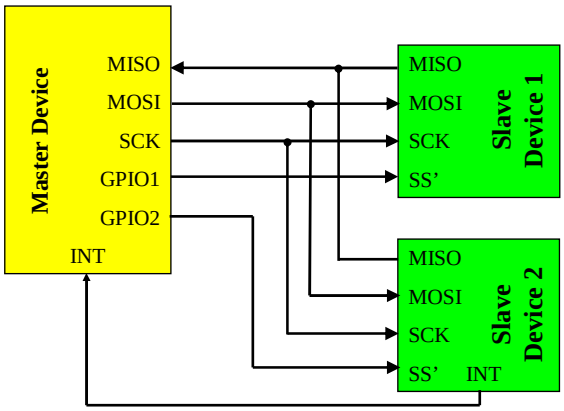
SPI Wiring Modes



4-Wire Single Master and 4-Wire Multiple Slave Mode Connection Diagram. This is the most frequent configuration.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

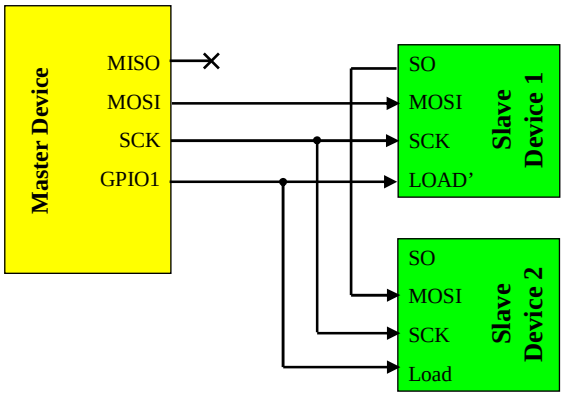
SPI Wiring Modes



4-Wire Single Master and 4-Wire Multiple Slave Mode Connection Diagram Using an Interrupt

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

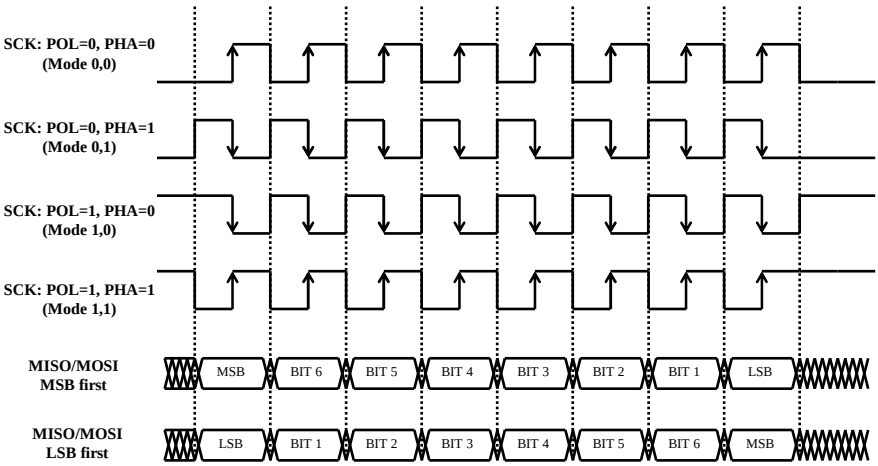
SPI Wiring Modes



4-Wire Single Master and Daisy Chained Multiple Slave Mode Connection Diagram

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

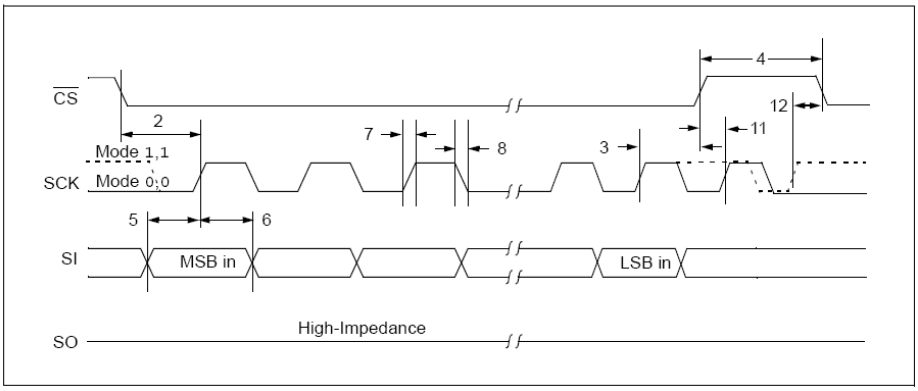
SPI Timing



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

25LC320 EEPROM Timing

FIGURE 1-2: SERIAL INPUT TIMING

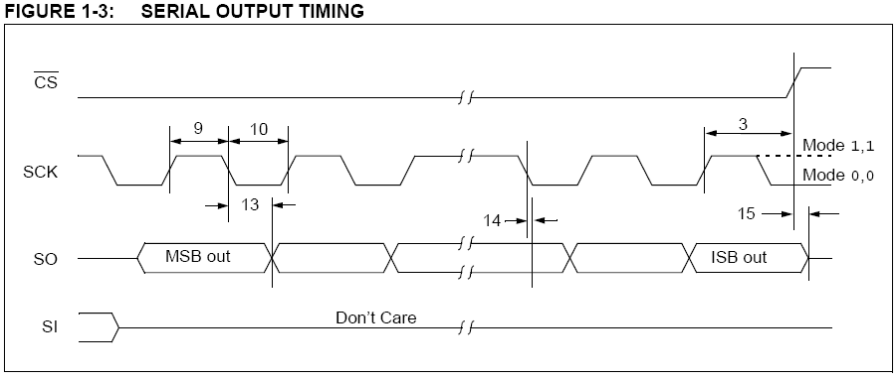


From 25AA320/25LC320/25C320 Datasheet. Copyright © Microchip

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

25LC320 EEPROM Timing

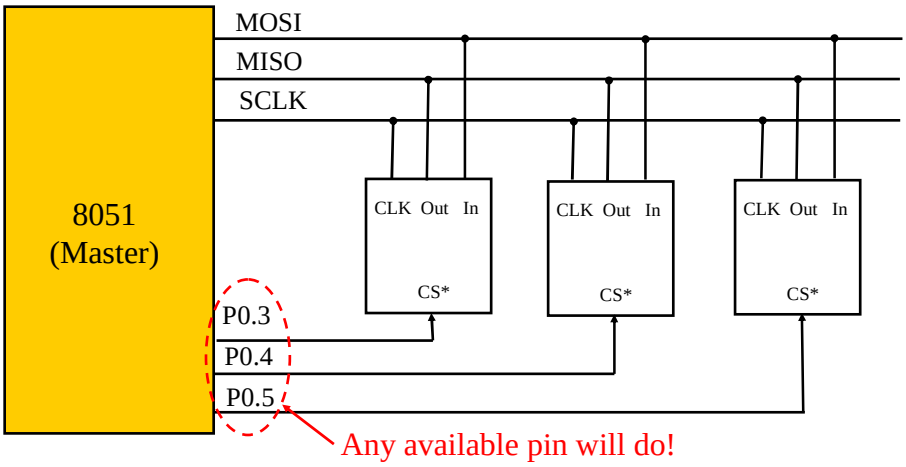
FIGURE 1-3: SERIAL OUTPUT TIMING



From 25AA320/25LC320/25C320 Datasheet. Copyright © Microchip

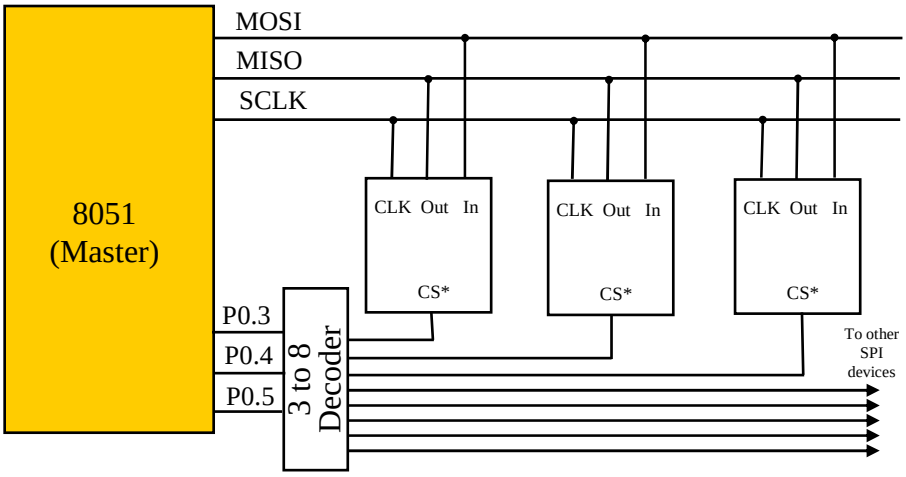
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Connecting Devices to the SPI port



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Connecting to the SPI port using a decoder



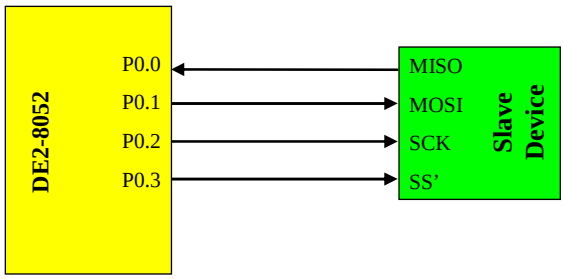
Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Bit Bang SPI

- Many modern microcontrollers have one or more built-in SPI controllers. This includes almost all Freescale (formerly Motorola), PICs from Microchip, 8051s derivatives from Silabs, Atmel, Dallas/Maxim, and NXP (formerly Philips), and ARMs from Atmel, NXP, and many others.
- Occasionally you may need to connect a SPI device to a microcontroller without an SPI controller. In this case we can **Bit Bang** SPI signals using GPIO pins!
- The DE2-8052 has not built-in SPI controller, so Bit Bang SPI it is!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Bit Bang SPI



GPIO pins arbitrarily chosen…

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Bit Bang SPI in Mode (0,0)

```
CE    EQU P0.3
SCLK  EQU P0.2
MOSI  EQU P0.1
MISO  EQU P0.0
DO_SPI:
    orl P0MOD, #00001110b ; Set CE, SCLK, MOSI as outputs
    anl P0MOD, #11111110b ; Set MISO as input
    clr SCLK              ; Mode 0,0 default
    clr CE                ; Enable device (active low)
    mov R1, #0            ; Received byte stored in R1
    mov R2, #8            ; Loop counter (8-bits)
DO_SPI_LOOP:
    mov a, R0             ; Byte to write is in R0
    rlc a                 ; Carry flag has bit to write
    mov R0, a
    mov MOSI, c
    setb SCLK             ; Transmit
    mov c, MISO           ; Read received bit
    mov a, R1             ; Save received bit in R1
    rlc a
    mov R1, a
    clr SCLK
    djnz R2, DO_SPI_LOOP
    setb CE               ; Disable device
    ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Bit Bang SPI: more general version

The caller must enable the slave device before calling this subroutine, and disable it after.

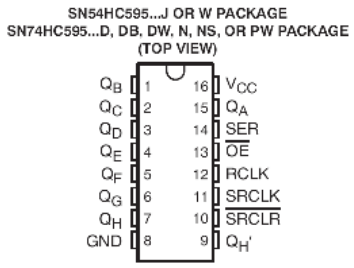
```
SCLK  EQU P0.2
MOSI  EQU P0.1
MISO  EQU P0.0

INI_SPI:
    orl P0MOD, #00000110b ; Set SCLK, MOSI as outputs
    anl P0MOD, #11111110b ; Set MISO as input
    clr SCLK              ; Mode 0,0 default
    ret

DO_SPI_G:
    mov R1, #0            ; Received byte stored in R1
    mov R2, #8            ; Loop counter (8-bits)
DO_SPI_G_LOOP:
    mov a, R0             ; Byte to write is in R0
    rlc a                 ; Carry flag has bit to write
    mov R0, a
    mov MOSI, c
    setb SCLK             ; Transmit
    mov c, MISO           ; Read received bit
    mov a, R1             ; Save received bit in R1
    rlc a
    mov R1, a
    clr SCLK
    djnz R2, DO_SPI_G_LOOP
    ret
```

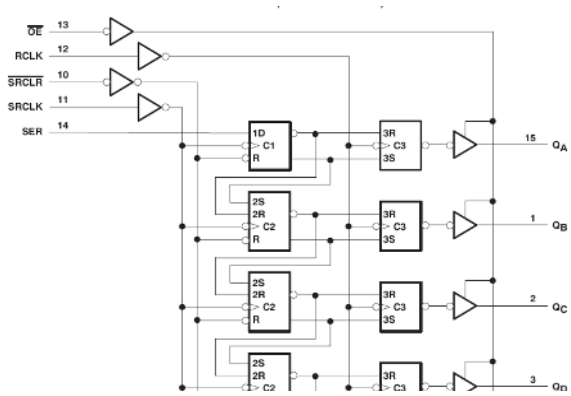
Mode (0,0)

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: Simple SPI Output Expansion Using the 74HC595

To understand the utility of SPI, let us expand the output ports of a DE2-8052 using an 8-bit shift register, the 74HC595:





Figures are from Texas Instruments' 74HC595 data sheet

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 1: Simple SPI Output Expansion Using the 74HC595

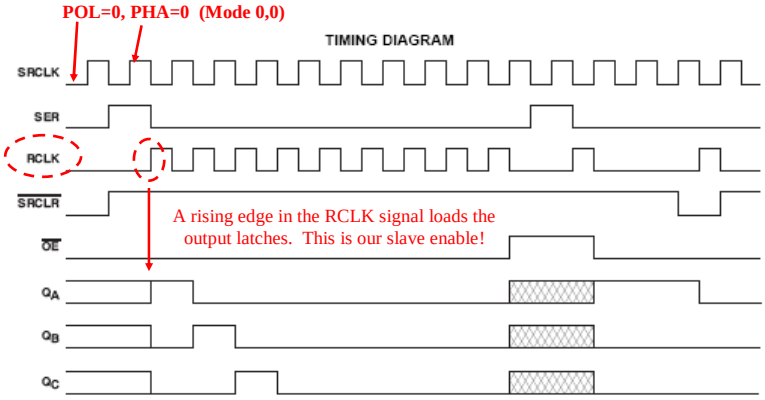
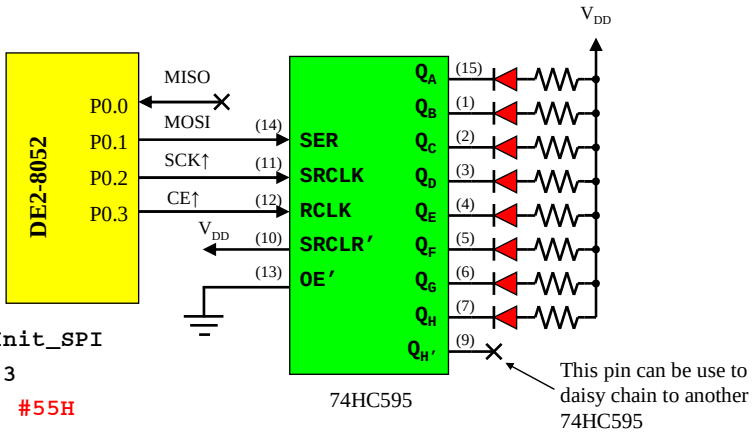


Figure is from Texas Instruments' 74HC595 data sheet

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

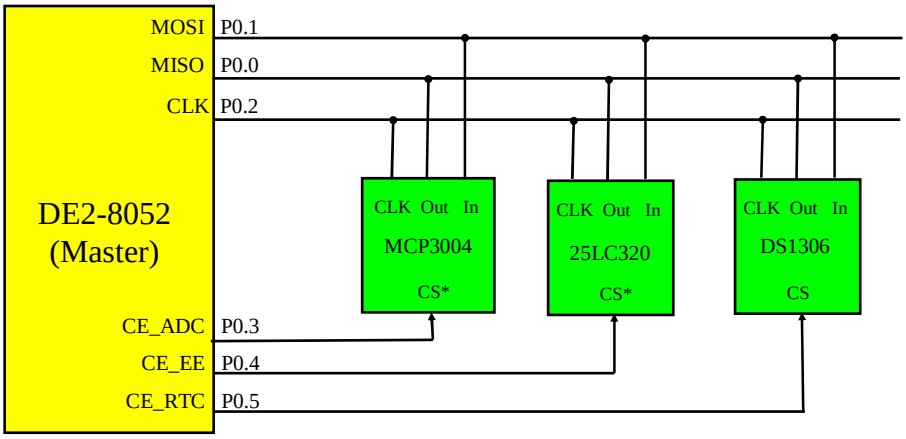
Example 1: Simple SPI Output Expansion Using the 74HC595




```
Lcall Init_SPI
Clr P0.3
Mov R0, #55H
Lcall DO_SPI_G
Setb P0.3
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Examples 2, 3, and 4: Connecting the MCP3004, 25LC320, and DS1306 to the same DE2-8052



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Examples 2, 3, and 4: Connecting the MCP3004, 25LC320, and DS1306 to the same DE2-8052

At the initialization of each example we must put all enables into the inactive state because the devices are all connected to the same SPI bus:

```
orl P0MOD, #00111000b ; make all CEs outputs

setb CE_ADC
setb CE_EE
clr  CE_RTC ; RTC CE is active high
```

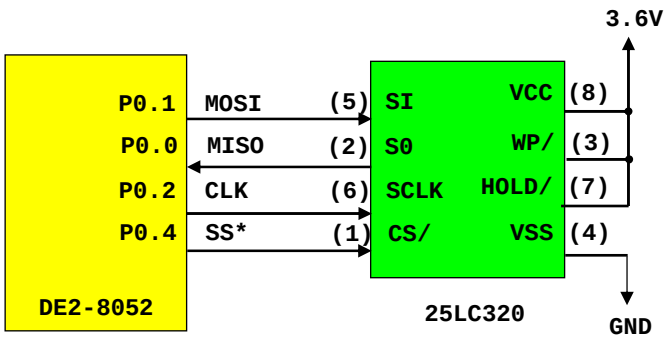

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 2 : Adding SPI EEPROM

- In this example I'll be using a 4k x 8 EEPROM from Microchip: 25LC320.
- Using SPI as four-wire master in mode 0,0.
- The 25LC320 requires command sequences of up to a 32 bits, therefore we may need to send 4 consecutive bytes.
- P0.4 used as chip enable for the 25LC320 EEPROM (active low).

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 2 : Adding a SPI EEPROM



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

EEPROM Instruction Set

TABLE 3-1: INSTRUCTION SET

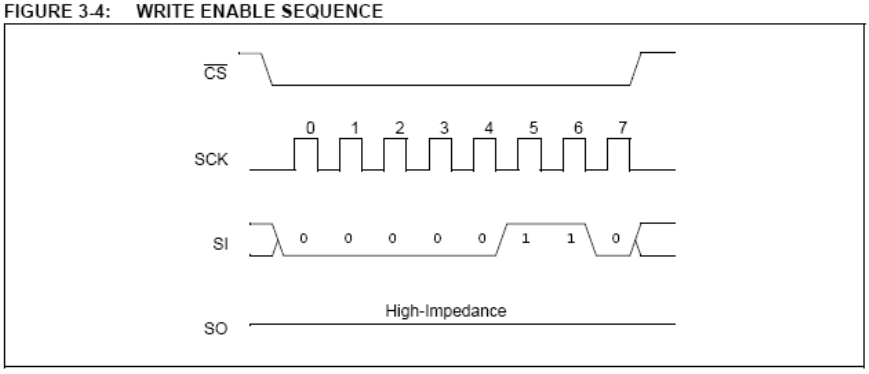
Instruction Name	Instruction Format	Description
READ	0000 0011	Read data from memory array beginning at selected address
WRITE	0000 0010	Write data to memory array beginning at selected address
WRDI	0000 0100	Reset the write enable latch (disable write operations)
WREN	0000 0110	Set the write enable latch (enable write operations)
RDSR	0000 0101	Read STATUS register
WRSR	0000 0001	Write STATUS register

From 25AA320/25LC320/25C320 Datasheet. Copyright © Microchip

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

EEPROM Enable Write Enable Latch Sequence

FIGURE 3-4: WRITE ENABLE SEQUENCE



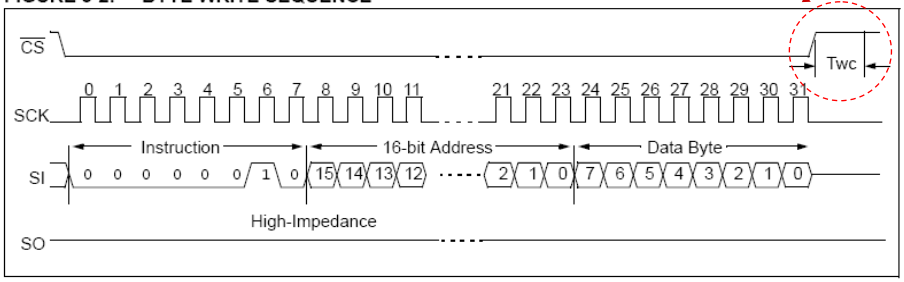
From 25AA320/25LC320/25C320 Datasheet. Copyright © Microchip

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

EEPROM Byte Write Sequence

Write time is 5ms!

FIGURE 3-2: BYTE WRITE SEQUENCE

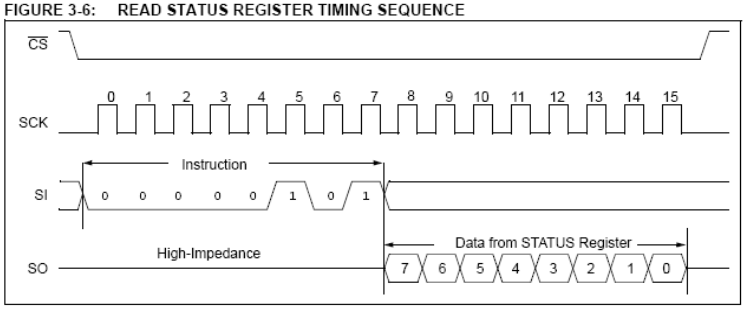


From 25AA320/25LC320/25C320 Datasheet. Copyright © Microchip

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

EEPROM Read Status Register Sequence

FIGURE 3-6: READ STATUS REGISTER TIMING SEQUENCE



WPEN	**X**	**X**	**X**	**BP1**	**BP0**	**WEL**	**WIP**

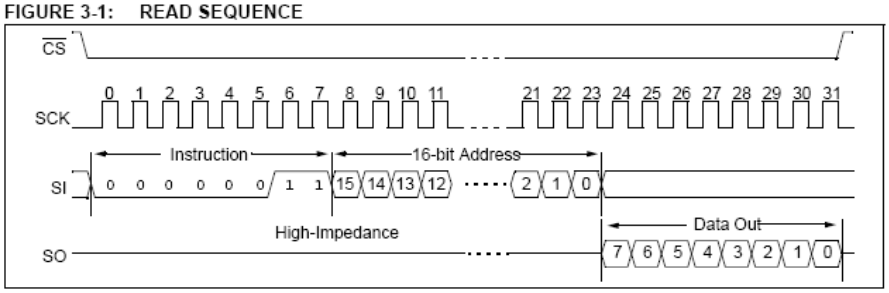
Status Register

From 25AA320/25LC320/25C320 Datasheet. Copyright © Microchip

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

EEPROM Byte Read Sequence

FIGURE 3-1: READ SEQUENCE



From 25AA320/25LC320/25C320 Datasheet. Copyright © Microchip

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Code: Definitions

```
$MODDE2
org 0000H
   ljmp MyProgram

CE    EQU P0.4
SCLK  EQU P0.2
MOSI  EQU P0.1
MISO  EQU P0.0

; Microchip 25LC320 Slave EEPROM Parameters and Instruction Set
EEPROM_CMD_WRSR   EQU  01H
EEPROM_CMD_WRITE  EQU  02H
EEPROM_CMD_READ   EQU  03H
EEPROM_CMD_WRDI   EQU  04H
EEPROM_CMD_RDSR   EQU  05H
EEPROM_CMD_WREN   EQU  06H

Delay:
    mov R3, #20
Delay_loop:
    djnz R3, Delay_loop
    ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Code: SPI routines

```
INIT_SPI:
    orl P0MOD, #00000110b ; Set SCLK, MOSI as outputs
    anl P0MOD, #11111110b ; Set MISO as input
    clr SCLK              ; For mode (0,0) SCLK is zero
    ret

DO_SPI_G:
    push acc
    mov R1, #0            ; Received byte stored in R1
    mov R2, #8            ; Loop counter (8-bits)
DO_SPI_G_LOOP:
    mov a, R0             ; Byte to write is in R0
    rlc a                 ; Carry flag has bit to write
    mov R0, a
    mov MOSI, c
    setb SCLK             ; Transmit
    mov c, MISO           ; Read received bit
    mov a, R1             ; Save received bit in R1
    rlc a
    mov R1, a
    clr SCLK
    djnz R2, DO_SPI_G_LOOP
    pop acc
    ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Code: Write to EEPROM

```
EEPROM_Write: ; DPTR: address, Acc: value to write
    ; Step 1: Set the Write Enable Latch to 1
    clr CE ; Activate EEPROM
    mov R0, #EEPROM_CMD_WREN
    lcall DO_SPI_G
    setb CE
    lcall Delay
    ; Step 2: Send the WRITE command
    clr CE
    mov R0, #EEPROM_CMD_WRITE
    lcall DO_SPI_G
    ; Step 3: Send the EEPROM destination address (MSB first)
    mov R0, DPH
    lcall DO_SPI_G
    mov R0, DPL
    lcall DO_SPI_G
    ; Step 4: Send the byte to write
    mov R0, A
    lcall DO_SPI_G
    setb CE ; Deactivate EEPROM after write
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Code: Write to EEPROM (cont.)

```
    ; Step 5: Poll on the Write In Progress (WIP) bit in the Stat. Reg.
EEPROM_Write_Loop:
    lcall Delay
    clr CE
    mov R0, #EEPROM_CMD_RDSR ; Send the Read Status Register command
    lcall DO_SPI_G
    mov R0, #0
    lcall DO_SPI_G ; Dummy write to output serial clock
    setb CE        ; Deactivate EEPROM after read
    mov a, R1
    anl a, #0x01
    jnz EEPROM_Write_Loop
    ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Code: Read from EEPROM

```
EEPROM_Read: ; DPTR: Address, Acc: Value read

    ; Step 1: Send the READ command
    clr CE ; Activate EEPROM
    mov R0, #EEPROM_CMD_READ
    lcall DO_SPI_G

    ; Step 2: Send the EEPROM destination address (MSB first)
    mov R0, DPH
    lcall DO_SPI_G
    mov R0, DPL
    lcall DO_SPI_G

    ; Step 3: Read the value returned
    mov R0, #0 ; Dummy write to output serial clock
    lcall DO_SPI_G
    setb CE ; Deactivate EEPROM
    lcall Delay

    mov a, R1
    ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Code: Main program

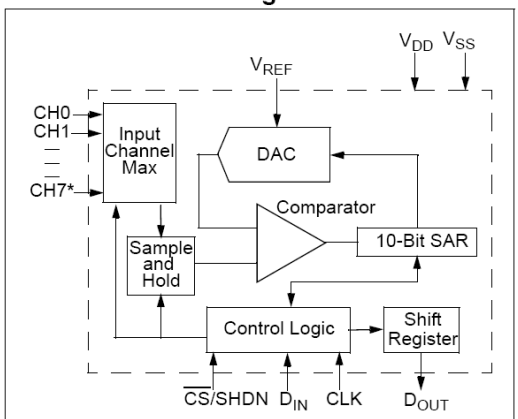
```
MyProgram:
    mov sp, #07FH
    clr a
    mov LEDG,  a
    mov LEDRA, a
    mov LEDRB, a
    mov LEDRC, a
    orl P0MOD, #00010000b ; make CE (P0.4) output

    lcall INIT_SPI

    mov dptr, #007BH
    mov a, #055H
    lcall EEPROM_Write ; Test: write 55H at address 007BH
    lcall EEPROM_Read ; Read back and compare

    cjne a, #055H, Did_not_work
    setb LEDG.7 ; It worked!
    sjmp Forever

Did_not_work:
    setb LEDRA.0

Forever:
    sjmp Forever
END
```

```
orl P0MOD, #00111000b ; make all CEs outputs

setb CE_ADC
setb CE_EE
clr  CE_RTC ; RTC CE is active high
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 3: Using the MCP3004 10-bit, 4-channel ADC

Functional Block Diagram

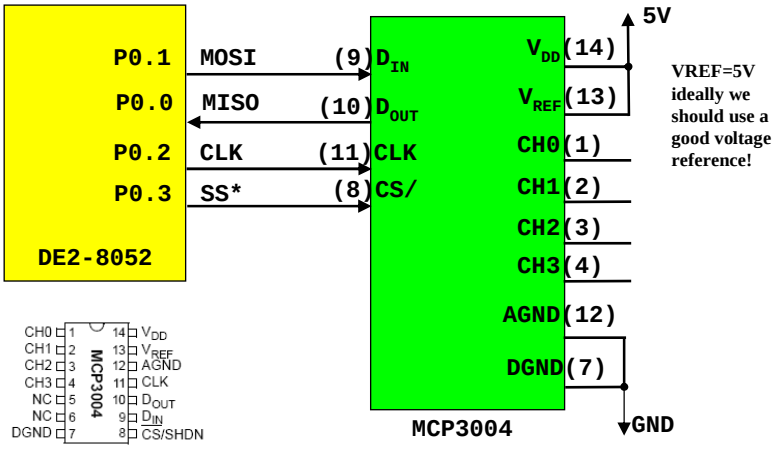


* **Note:** Channels 4-7 are available on MCP3008 Only

Figure taken From Microchip's MCP3008 datasheet

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 3: Using the MCP3004 10-bit, 4-channel ADC



VREF=5V ideally we should use a good voltage reference!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 3: Using the MCP3004 10-bit, 4-channel ADC

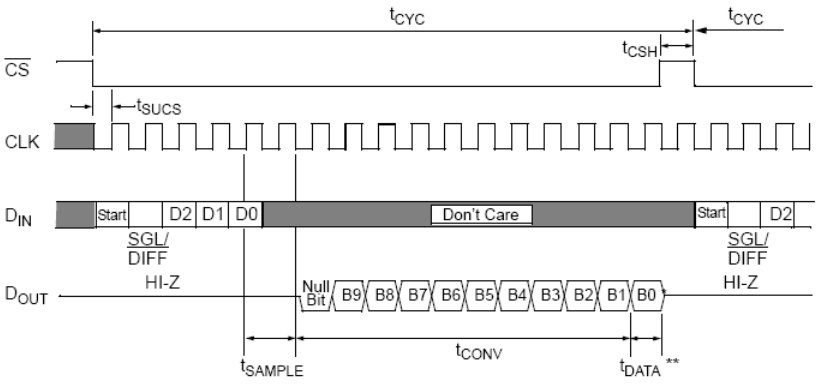


Figure taken from Microchip's MCP3008 datasheet

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 3: Using the MCP3004 10-bit, 4-channel ADC

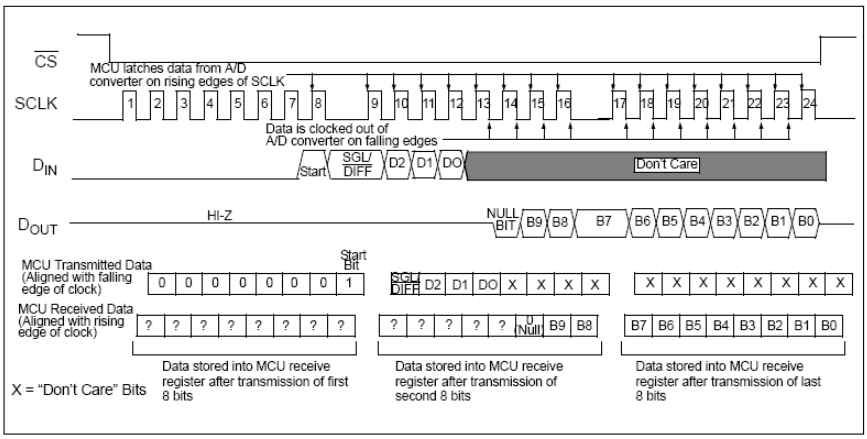


FIGURE 6-1: *SPI Communication with the MCP3004/3008 using 8-bit segments (Mode 0,0: SCLK idles low).*

Figure taken from Microchip's MCP3008 datasheet

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

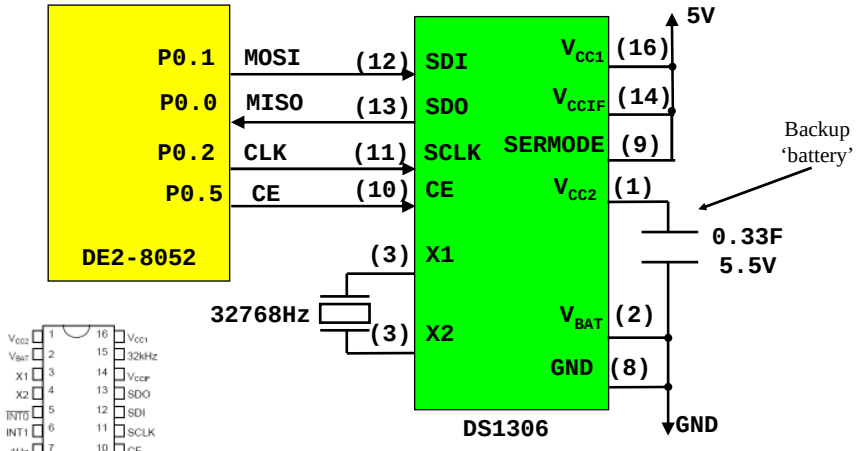
Example 3: Using the MCP3004 10-bit, 4-channel ADC

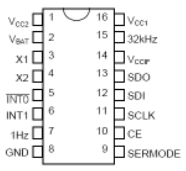
```
    CE_ADC EQU  P0.3
    orl P0MOD, #00001000b ; make CE_ADC (P0.3) output
    lcall INIT_SPI

Forever:
    clr CE_ADC
    mov R0, #00000001B ; Start bit:1
    lcall DO_SPI_G

    mov R0, #10000000B ; Single ended, read channel 0
    lcall DO_SPI_G
    mov a, R1          ; R1 contains bits 8 and 9
    anl a, #0FH
    mov LEDRB, a

    mov R0, #55H       ; It doesn't matter what we transmit...
    lcall DO_SPI_G
    mov LEDRA, R1      ; R1 contains bits 0 to 7
    setb CE_ADC
    lcall Delay

    sjmp Forever
```

```
orl P0MOD, #00111000b ; make all CEs outputs

setb CE_ADC
setb CE_EE
clr  CE_RTC ; RTC CE is active high
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 4: Real Time Clock DS1306





Warning: the 1Hz output needs a pull-up resistor.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 4: Real Time Clock DS1306

```
INIT_SPI:
    orl P0MOD, #00000110b ; Set SCLK, MOSI as outputs
    anl P0MOD, #11111110b ; Set MISO as input
    clr SCLK              ; For mode (0,0) SCLK is zero
    ret

DO_SPI_G:
    push acc
    mov R1, #0            ; Received byte stored in R1
    mov R2, #8            ; Loop counter (8-bits)
DO_SPI_G_LOOP:
    mov a, R0             ; Byte to write is in R0
    rlc a                 ; Carry flag has bit to write
    mov R0, a
    mov MOSI, c
    setb SCLK             ; Transmit
    nop                   ; DE2-8052 @ 33.33MHz is too fast so...
    nop
    mov c, MISO           ; Read received bit
    mov a, R1             ; Save received bit in R1
    rlc a
    mov R1, a
    clr SCLK
    djnz R2, DO_SPI_G_LOOP
    pop acc
    ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 4: Real Time Clock DS1306

Figure 2. RTC REGISTERS AND ADDRESS MAP

HEX ADDRESS READ	HEX ADDRESS WRITE	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	RANGE
00h	80h	0	10 SEC			SEC				00–59
01h	81h	0	10 MIN			MIN				00–59
02h	82h	0	12	P/A	10-HR	HOURS				01–12 + P/A
			24	10						00–23
03h	83h	0	0	0	0	0	DAY			01–07
04h	84h	0	0	10-DATE		DATE				1–31
05h	85h	0	0	10-MONTH		MONTH				01–12
06h	86h	10-YEAR				YEAR				00–99
07h	87h	M	10-SEC ALARM 0			SEC ALARM 0				00–59
08h	88h	M	10-MIN ALARM 0			MIN ALARM 0				00–59
09h	89h	M	12	P/A	10-HR	HOUR ALARM 0				01–12 + P/A
			24	10						00–23

Figure taken from Maxim/Dallas DS1306 datasheet

All BCD!

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 4: Real Time Clock DS1306

0Ah	8Ah	M	0	0	0	0	DAY ALARM 0	01–07
0Bh	8Bh	M	10 SEC ALARM 1			SEC ALARM 1		00–59
0Ch	8Ch	M	10 MIN ALARM 1			MIN ALARM 1		00–59
0Dh	8Dh	M	12	P	10-HR	HOUR ALARM 1		01–12 + P/A
				A				
			24	10				00–23
0Eh	8Eh	M	0	0	0	0	DAY ALARM 1	01–07
—								
0Fh	8Fh	CONTROL REGISTER						—
10h	90h	STATUS REGISTER						—
11h	91h	TRICKLE CHARGER REGISTER						—
12h–1Fh	92h–9Fh	RESERVED						—
20h–7Fh	A0h–FFh	96-BYTES USER RAM						—

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 4: Real Time Clock DS1306

Figure 6. SPI SINGLE-BYTE WRITE

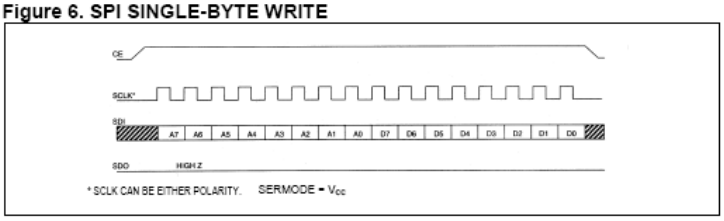
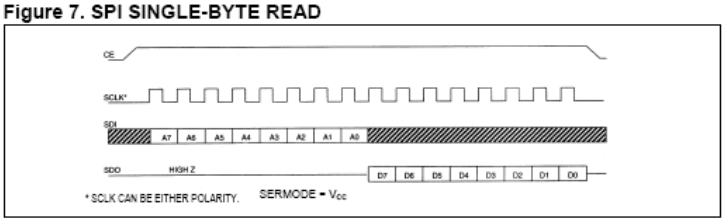


Figure 7. SPI SINGLE-BYTE READ



Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 4: Real Time Clock DS1306

```
; dpl has the address, b has the data
RTC_Write:
        setb   CE_RTC
        mov    a, #0x80
        orl    a, dpl
        mov    R0, a
        lcall  DO_SPI_G
        mov    R0, b
        lcall  DO_SPI_G
        clr    CE_RTC
        ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Example 4: Real Time Clock DS1306

```
; dpl has the address, value returned in accumulator
RTC_Read:
        setb   CE_RTC
        mov    a, #0x7F
        anl    a, dpl
        mov    R0,a
        lcall  DO_SPI_G
        mov    R0, #0x00 ; Any value will do
        lcall  DO_SPI_G
        clr    CE_RTC
        mov    a, R1
        ret
```


Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.

Exercises

- Modify the functions in slide 22 so that the SPI is operated in mode (1,1).
- Show how to add four 25LC320 SPI EEPROMs to the DE2-8052. Use P1.0, P1.4, P1.5, and P1.6 as chip enables.
- Show how to connect two 74HC595 ICs to the SPI port using a daisy chain configuration.
- The DS1302 is a Real Time Clock IC that uses a serial interface similar to SPI. Unlike SPI that uses separated pins for transmission and reception, the DS1302 uses a bidirectional pin for both transmission and reception. Referring to figure 4 of the data sheet for the DS1302 (http://pdfserv.maxim-ic.com/en/ds/DS1302.pdf) write a serial transmission subroutine and a serial reception subroutine for this RTC in assembly language for the DE2-8052.

Copyright © 2009-2014, Jesus Calvino-Fraga. Not to be copied, used, or revised without explicit written permission from the copyright owner.